NumCompute: A High-Performance NumPy Toolkit

Assignment 2.1: Programming Task 1 — Technical Report

Repository: https://github.com/bookmountain/numcompute-review

Name	Student ID	Area	Branch
Katherine Roman		Data I/O and preprocessing	`katherine/data-preprocessing`
Shushan Chang	a1951976	Sorting, searching and ranking	`book/algorithm`
Lachlan McBrearty		Statistics and metrics	`stats-metrics`
Ruofan Li		Pipelines, optimisation, utilities and benchmarking	`raven/composition`

1. Design choices (vectorisation and API)

NumCompute is written in plain Python and NumPy only. Each module owns one area of the toolkit, and modules share input checks through `numcompute/_validation.py` so that type and shape errors are reported consistently.

1.1 Data I/O and preprocessing (Katherine Roman)

1.2 Sorting, searching and ranking (Shushan Chang)

My part of NumCompute is `sort_search.py` and `rank.py`. All functions take NumPy-compatible numeric input, return new results without modifying the caller's data, and use the shared `_validation.py` helpers so errors are consistent across modules. One convention to watch: k is a count in `topk` but a zero-based position in `quickselect`.

`stable_sort` wraps `numpy.sort(kind="stable")`, so equal values keep their input order. `multi_key_sort` uses `numpy.lexsort`. Because `lexsort` treats the last column it is given as the most important, the function reverses the user's column list first.

`topk` avoids sorting the whole array. `numpy.argpartition` finds the k-th best value, which acts as a cutoff. Every value better than the cutoff is kept, and any places still free are filled with values equal to the cutoff, earliest first, so ties always resolve the same way. Only these k values are then sorted, giving O(n + k log k) time instead of O(n log n).

`quickselect` is a hand-written version of the same idea. It picks a pivot, splits the values into those below, equal to and above it, and keeps only the group that contains position k. It repeats this until position k falls on the pivot. This takes expected O(n) time, with an $O(n^2)$ worst case. The repetition needs a Python loop because each step uses the previous step's result, but the splitting inside each step is vectorised.

`binary_search` repeatedly halves the search range of a sorted array and returns the leftmost insertion index and whether the value exists, in O(log n) time. It does not check whether the array is sorted or contains NaN or infinite values, because that would mean reading every element and cost O(n). Providing valid input is left to the caller, as the docstring states.

`rank` sorts the non-NaN values, groups values that are equal, and assigns ordinal, dense or average ranks. I added `ascending=False` so that the highest exam score can be rank 1. `percentile` validates its arguments and then calls NumPy, supporting four interpolation methods, calculation along an axis, and three NaN policies.

Operation	Technique	Time complexity	Key contract
Stable/multi-key sort	Stable sort/`lexsort`	`O(n log n)`	Stable ties; first listed column is most important
Top-k	Partial partition	`O(n + k log k)`	Ties prefer earlier indexes
Quickselect	Three-way partitioning	Expected `O(n)`	k is zero-based
Binary search	Interval halving	`O(log n)`	Input is already sorted and finite
Ranking/percentile	Sorting and interpolation	`O(n log n)` upper bound	Explicit tie, direction and NaN rules

Table 1. Techniques, complexity and contracts for sorting, searching and ranking.

1.3 Statistics and metrics (Lachlan McBrearty)

stats.py provides axis-aware mean, median, variance, standard deviation, minimum/maximum, histograms and quantiles, plus mergeable Welford/Chan RunningStats. Reductions support axis and tuple axes, keepdims and propagate/omit/raise NaN policies. Mean returns float64; variance divides the squared-deviation sum by count minus ddof and returns NaN with a RuntimeWarning when a slice has no more valid observations than ddof.

metrics.py implements confusion matrix (true labels in rows, predictions in columns), accuracy, precision, recall, F1, mean squared error, ROC curves and AUC. Classification metrics support explicit labels and documented binary, micro, macro and weighted averages where applicable. Two current behavior decisions are recorded in API_CONTRACT.md: integer ROC thresholds may use object dtype to preserve exact values, and weighted F1 with zero_division=np.nan may propagate NaN for an included zero-support class.

1.4 Pipelines, optimisation and utilities (Ruofan Li)

These modules use Python and NumPy. `grad` and `jacobian` support forward, backward and central finite differences. For n inputs, one-sided schemes require n + 1 function evaluations; central schemes require 2n for gradients and 2n + 1 for Jacobians. For smooth functions, one-sided schemes have O(h) truncation error and central schemes have $O(h^2)$ truncation error. Validation rejects invalid arguments, inconsistent output shapes, ineffective perturbations and non-finite inputs, evaluations or derivatives, including overflow. Float64 copies preserve caller inputs.

`Transformer` and `Estimator` protocols define composition interfaces. `Pipeline` fits steps sequentially, forwards targets only to a final estimator, and reuses fitted state for transformation or prediction. `FeatureUnion` fits branches on independent input copies and concatenates features in branch order. Containers check fitted state, unique nonempty names, callable methods, two-dimensional arrays and row preservation. Step objects are fitted directly. There is no global fixed feature-count restriction: `n_features_in_` is metadata; compatibility belongs to individual steps.

`sigmoid uses sign-dependent evaluation; relu applies an elementwise maximum. softmax and logsumexp subtract maxima before exponentiation; the latter handles empty and non-finite slices. Python-integer temporaries preserve integer differences, with a rounding-residual correction in logsumexp. euclidean_distance uses np.hypot.reduce to avoid unnecessary overflow; manhattan_distance sums absolute differences. Two integer distance inputs are subtracted before float conversion, avoiding wraparound and lost gaps. iter_batches yields ordered views whose modification can affect the source. The duplicate top_k_indices was removed; integrated sort_search.topk is the project's single top-k implementation.`

Below, m is derivative output size, N input elements, Q reduction output elements, N_i pipeline intermediate sizes, b branches, M concatenated elements and B batches. C and R denote benchmark cases and repeats.

Component	Technique/design	Time complexity	Key contract
`grad` / `jacobian`	Coordinate differences	$O(n^2)$ / $O(n^2 + mn)$, plus evaluations	Outputs (n,) / (m, n); space $O(n)$ / $O(mn + n + m)$, excluding callable allocations
`Pipeline`	Sequential composition	Step costs + $O(\Sigma N_i)$	Extra array storage $O(\max N_i)$, excluding step internals/outputs; array-target copying adds $O(y.size)$
`FeatureUnion`	Branch concatenation	Branch costs + $O(bN + M)$ for transformation	Array storage $O(N + M)$, excluding branch internals
Numerical utilities	Vectorised operations	Elementwise $O(N)$; reductions $O(N + Q)$	Float64 outputs; space $O(N + Q)$ for reductions; equal-shaped distance inputs
`iter_batches`	Consecutive slices	$O(B)$	Auxiliary space $O(1)$ at fixed rank, excluding retained batches
Benchmark harness	Repeated measurements	Proportional to calls and workload cost, plus checks	Report storage $O(CR)$, excluding workloads

Table 2. Techniques, complexity and contracts for pipelines, optimisation and utilities.

2. Testing and edge cases

2.1 Data I/O and preprocessing

2.2 Sorting, searching and ranking

Fourteen unit tests compare each function's output with answers worked out by hand, including all ranking methods in both directions and every interpolation method. Edge cases cover empty arrays, extreme k, all-equal values, ties at the top-k cutoff, duplicates, NaNs, non-contiguous views, large `uint64` values, boolean input, and checks that inputs are not modified. All tests pass.

NaN handling differs on purpose. `topk` and `quickselect` reject NaN because it has no meaningful position in sorted order, `binary_search` expects sorted data with no NaN or infinite values, and `rank` and `percentile` let the caller choose to omit, propagate or raise. The quickstart therefore has to filter or impute missing CSV values before using selection or search.

2.3 Statistics and metrics

Statistics and metrics tests cover axes and output shapes, NaN policies and all-NaN slices, ddof validation and exhausted degrees of freedom, streaming updates and merges, histogram/quantile behavior, confusion-matrix label order, undefined precision/recall/F1, multiclass averaging, ROC score ties and invalid inputs. Regression tests also cover mixed signed/unsigned labels, large integer score ordering, small benchmark row counts and subnormal variance/std results.

The default and supported four-row benchmark workloads each passed all 20 correctness checks.

2.4 Pipelines, optimisation and utilities

The complete integrated suite passed 642 tests, including parameterised cases; four warnings report the optional PyYAML dependency is unavailable. Tests cover analytic derivatives across all schemes, rectangular Jacobians, invalid arguments/outputs, ineffective steps and overflow; composition order, target forwarding, prediction, nesting, changed widths, copy isolation and failed refits; and utility axes, empty arrays, non-contiguous inputs, NaNs, infinities, extreme values, large signed/unsigned integers and batch views. Four harness tests cover execution counts/order, correctness failure, configuration validation and mocked-clock arithmetic without performance thresholds.

Finite differences remain sensitive to step size and roundoff; callables must not mutate evaluation inputs. Results remain limited by float64 range. Object-array copies are shallow; steps control dtype, missing-value and feature policies. Compatibility with actual scalers, imputers and encoders requires integration testing.

3. Benchmark results

Benchmarks exclude preparation and correctness checks, including batch contents/order and input preservation, from timing. Warmups precede repeated calls measured with perf_counter_ns; elapsed nanoseconds become seconds per call. Comparison order alternates, and raw samples plus minimum, median and maximum timings are retained. benchmark/results.json contains a recorded and verified benchmark snapshot: 20 workloads, seed 2026, one warmup, five repeats and three calls per sample. It records shapes, tolerances, configuration and environment metadata; all eight recorded source hashes match.

Workload	Shape	Python loop (ms)	NumPy (ms)	Speed-up
`utils.sigmoid`	2000×16	5.012	0.644	7.8×
`utils.relu`	2000×16	3.066	0.017	180.4×
`utils.softmax`	2000×16	7.410	0.181	40.9×
`utils.logsumexp`	2000×16	6.088	0.187	32.5×
`utils.euclidean_distance`	2000×16	4.979	0.449	11.1×
`utils.manhattan_distance`	2000×16	5.383	0.050	108.3×

Table 3. Median milliseconds per call from benchmark/results.json (Python 3.14.7, NumPy 2.5.3, macOS arm64, 8 CPUs; seed 2026, 5 repeats × 3 calls). Ratios apply to this environment only.

The integrated benchmark set covers utilities, differentiation, pipeline composition, statistics and metrics. Utilities, statistics and metrics use correctness-checked loop or pairwise baselines; derivatives use untimed analytic checks. Composition uses direct utility comparisons and adapters that learn no parameters, measuring overhead rather than preprocessing training. Timings concern one environment and finite float64 workloads; preprocessing, sorting/searching, ranking and I/O do not currently have benchmark cases. The integrated CSV pipeline smoke test and Quickstart script, notebook checker and fresh AIML-kernel notebook execution passed.

4. Reflections on teamwork

Member	Owned work	Review of another member
Katherine Roman	`io.py`, `preprocessing.py`, CSV/preprocessing tests, sample data, preprocessing demo	
Shushan Chang	`sort_search.py`, `rank.py`, sorting/ranking tests, report assembly	
Lachlan McBrearty		
Ruofan Li	`pipeline.py`, `optim.py`, `utils.py`, `benchmarking.py`, benchmark runner, tests	

Table 4. Contribution summary; evidence is recorded in CONTRIBUTIONS.md.

AI use

Member	AI tool and how it was used
Katherine Roman	
Shushan Chang	Claude Code was used to assemble and format this report.
Lachlan McBrearty	
Ruofan Li	